ESIGELEC
INGÉNIEUR.E.S GÉNÉRALISTES
SYSTÈMES INTELLIGENTS ET CONNECTÉS

# Technical Project

Group Class: **G2**

Team Number: **01**

Group Members:

**Zhao Hangjian (202544060167)**

**Zhou Hangyu (202544060168)**

**Jia Zihan(202544060121)**

September 2025

# 1. Project Objectives

Describe the objectives of the project with your own words.

This project employs both a sensor system and a camera vision system to achieve autonomous line-following and obstacle-avoidance parking functions for the Raspberry Pi car.

# 2. Project Schedule

Add here the schedule used during the project.

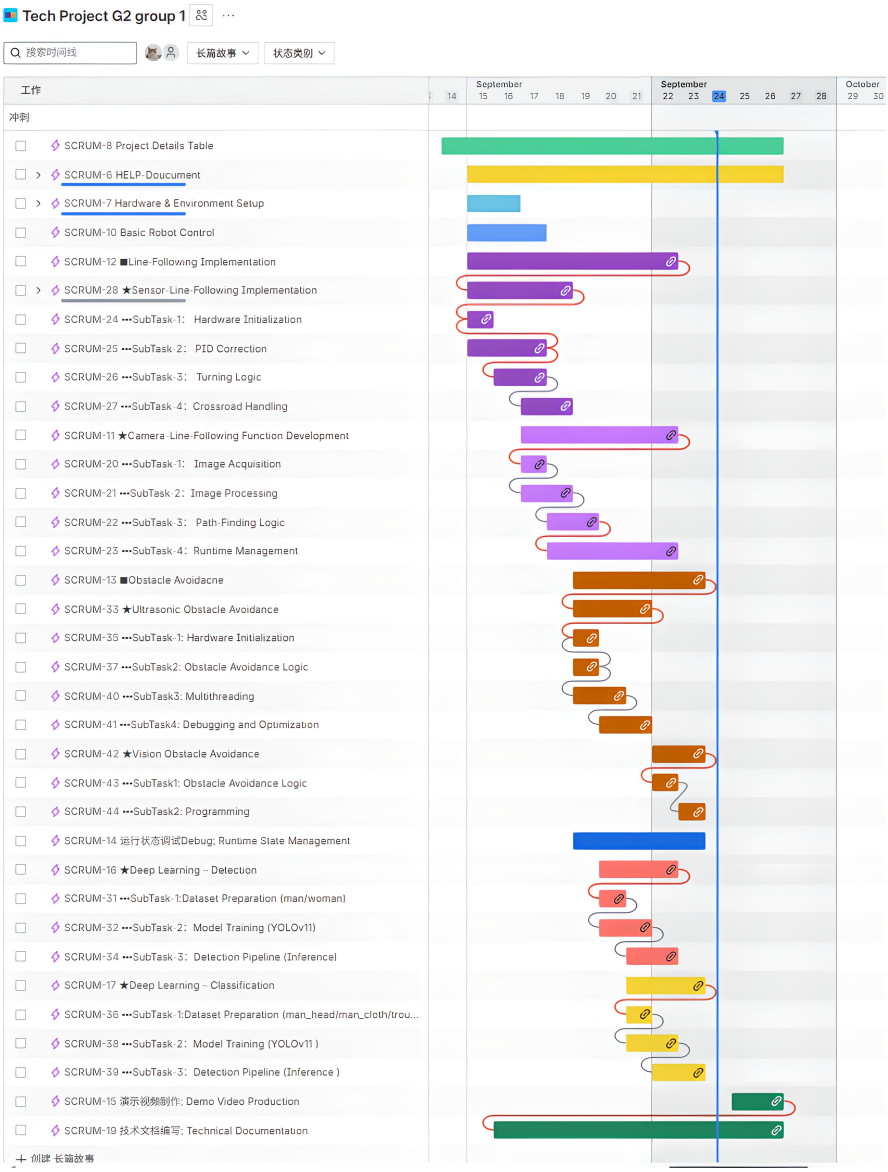
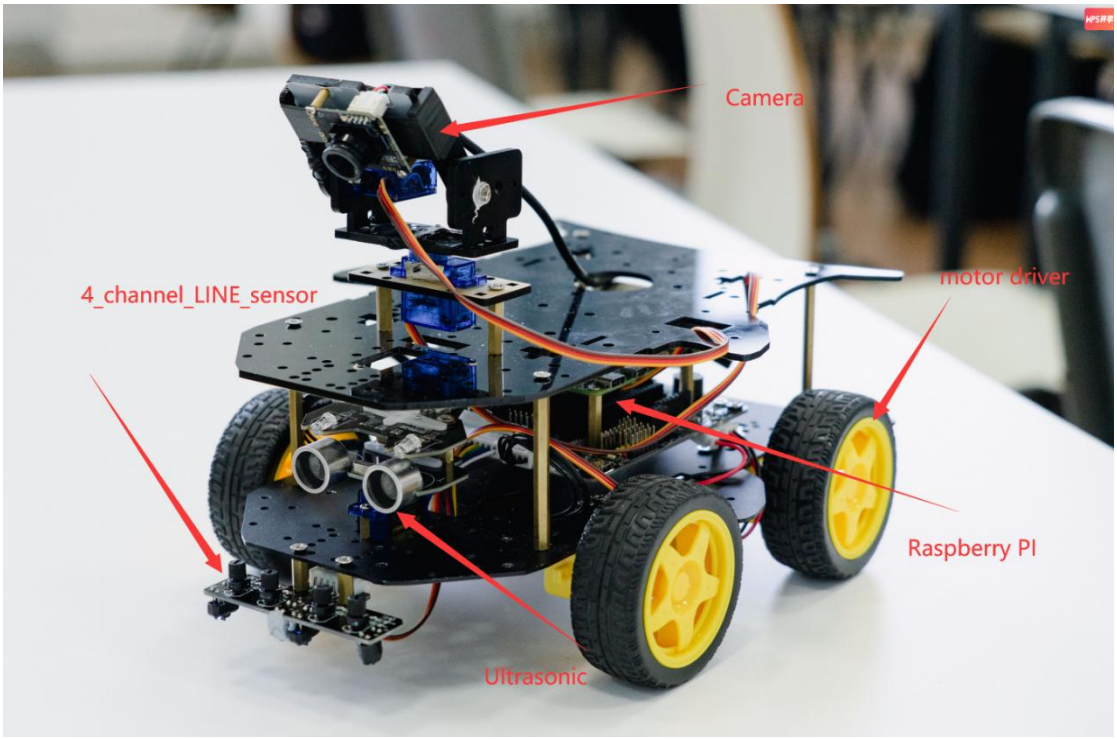



Figure [1] Gantt chart

# 3. Hardware

We removed the panels on the second floor of the car and deployed the camera in front to obtain a clearer front view. Since the project does not consider using the detector in front of the wheel, it was removed.

## 3.1 Hardware – Components Used




| Category | Components |
|---|---|
| Main Controller | Raspberry Pi |
| Sensors | - 4-channel line-following sensors<br>- Ultrasonic obstacle avoidance module |
| Actuators | DC motors (with motor driver) |
| Vision | Camera module (front-mounted) |
| Others | Power supply, wiring, GPIO connections |

# 4. Software

This project implements two sets of route recognition and obstacle avoidance algorithms, which are ① using OpenCV vision for road steering direction recognition and steering logic judgment, YOLO algorithm for obstacle recognition, and NumPy for matrix operations; ② using infrared sensors for line detection and ultrasonic sensors for obstacle detection.

## 4.1 Software – Reference Codebases & Dependencies

| Category | Libraries / Frameworks | Notes |
|---|---|---|
| Computer Vision | OpenCV | Image preprocessing, line detection, transformations |
| Numerical Computation | NumPy | Matrix operations, data processing |
| GPIO Control | RPi.GPIO | Motor and sensor input/output control |
| Deep Learning | YOLOv11 (Ultralytics Docs) | Object detection and classification, PyTorch-based |

# 5. Line Follow（sensor）

## a. Sensors

Using four circuit sensors, the 16 kinds of return values of the sensors are used to judge the path analogy and deduce the required operations.

## b. Algorithm

**5.1At the heart of this code is a proportional control (P control) algorithm.**

In the main loop, the system calculates the positional error (Error) between the vehicle body and the black line by reading the status of four infrared sensors. This error is then multiplied by a proportional constant Kp to generate a speed correction value. The program subsequently dynamically adjusts the speed difference between the left and right wheels (accelerating one while decelerating the other) based on this correction value, enabling automatic steering that keeps the vehicle aligned with the track. The algorithm also handles special cases such as sharp turns and route loss scenarios.

## c. Source Code

### 5.2Implementation of Infrared Sensor-Based Line Tracing Algorithm

This code demonstrates a four-channel infrared sensor-based path-following control system. By interpreting digital signals (0 or 1) from the sensors to identify the black line path, it employs a **Proportional Control Algorithm (P Control)** to dynamically adjust motor speed in real time, enabling autonomous path tracing for Raspberry Pi robots. The complete workflow encompasses hardware initialization, sensor data acquisition, status detection, error calculation, and motor control. The code is primarily divided into two sections:

- Hardware Initialization and Motion Control Module (init() and motion functions): Handles GPIO pin configuration, PWM initialization, and encapsulation of low-level motor motion control.
- Main Logic Module (while True loop): Responsible for cyclically reading sensors, detecting current states (straight driving, turning, line loss, etc.), calculating offset errors, and adjusting dual-wheel differential to achieve trajectory following.

### 5.2.1Hardware Initialization and Motion Control Module

This module is the physical basis of the tracking system, responsible for initializing hardware interfaces and providing simple motion instructions to support the upper-layer logic algorithm.

**5.2.1.1 init() function:**

1) Pin configuration: Set all GPIO pins connected to the motor driver board and sensors to the correct input/output modes (GPIO.setmode,GPIO.setup).
2) PWM initialization: Create a PWM object for the enable pins (ENA, ENB) controlling motor speed, configure the frequency to 2000Hz, and initiate it with a 0 duty cycle (stop state).

**5.2.1.2 Motion functions (e.g., run, spinLeft, etc.):**

The system encapsulates low/high-level GPIO signals into high-level, user-friendly action commands. For example, the spin_right() function enables a rightward turn by setting one motor to rotate clockwise while the other runs counterclockwise. The main logic module simply calls these functions with speed parameters, eliminating the need to handle specific pin operations.

### (1) Main logic module (main loop)

Responsible for coordinating the entire trace process, from reading sensor data to output motor control commands.

① **Initialization configuration**

Before entering the main cycle, define the core control parameters of the algorithm:

1) Base Speed (base_speed): Sets the baseline speed for the robot's path following.
2) Proportional Constant (Kp): A core parameter of the P controller that determines the correction intensity for route deviations.
3) Last Error Record (last_error): Stores the final steering position before line loss to assist the robot in recovering its navigation path.

② **Sensor status reading**

1) Cyclic data acquisition: During each iteration cycle, real-time digital signals from four track-following sensors (Left 1, Left 2, Right 1, Right 2) are collected.
2) Data format: The four detected values (where 0 indicates black line and 1 represents white background) are stored in a list called sensors, for example [1,0,0,1].

③ **Error calculation and path state analysis**

1) Status determination: The position of the car relative to the black line is determined by analyzing the combination mode of the sensor list.
2) Error quantization: The position state is quantized as a numerical error. This error value is equivalent to the "center of mass offset" in machine vision solutions.
   - error = 0: Perfectly centered.
   - error = 1 or 2: The car is off to the left and needs to be adjusted to the right.
   - error = -1 or-2: The car is off to the right and needs to be adjusted to the left.
   - The absolute value of the number indicates the severity of the deviation.

④ **Status machine and control strategy**

Based on the sensor combination, the system switches between different states and implements corresponding strategies.

| (Mode/State) | (Detection Condition) | (Turn Type) | (Control Strategy) | (Speed Parameters) | (Exit Condition) |
|---|---|---|---|---|---|
| P-ratio tracing | A single sensor is pressed on the inside or outside | Proportional fine-tuning of turns | P ratio algorithm correction | base_speed ± (Kp * error) | Continue, or enter another state |
| Left sharp turn | The left two sensors press the line at the same time [0,0,X,X] | spot turn to L | Call spinLeft() to perform a fixed action | spin_left(40, base_speed + 15) | The sensor status no longer meets the condition |
| Right turn | The right two sensors are pressed at the same time [X, X, 0, 0] | spot turn to R | Call spin_right() to perform a fixed action | spin_right(base_speed + 15, 40) | The sensor status no longer meets the condition |
| Crossroads/End point | All four sensors are wired [0,0,0,0] | Straight through | Call run() straight for a short time | run(base_speed, base_speed) | Exit automatically after a short delay |
| Loss of line status | All four sensors are on a white background [1,1,1,1] | Memory-based routing | Turn in the direction of the last error value to look for the loss of line | spin_left(30, base_speed) or spin_right(base_speed, 30) | The line was detected again |

Table [1] Turning strategies

⑤ **Speed correction and execution**

1) Correction calculation: Multiply the quantized error (error) by the proportional constant Kp to obtain the final speed correction value: correction = Kp × error.

2) Differential calculation: The system calculates the corrected speed by adding the correction value to one wheel's speed while subtracting it from the other, thereby generating differential force for steering.

- Left speed = base speed + correction
- Right speed = base speed-correction

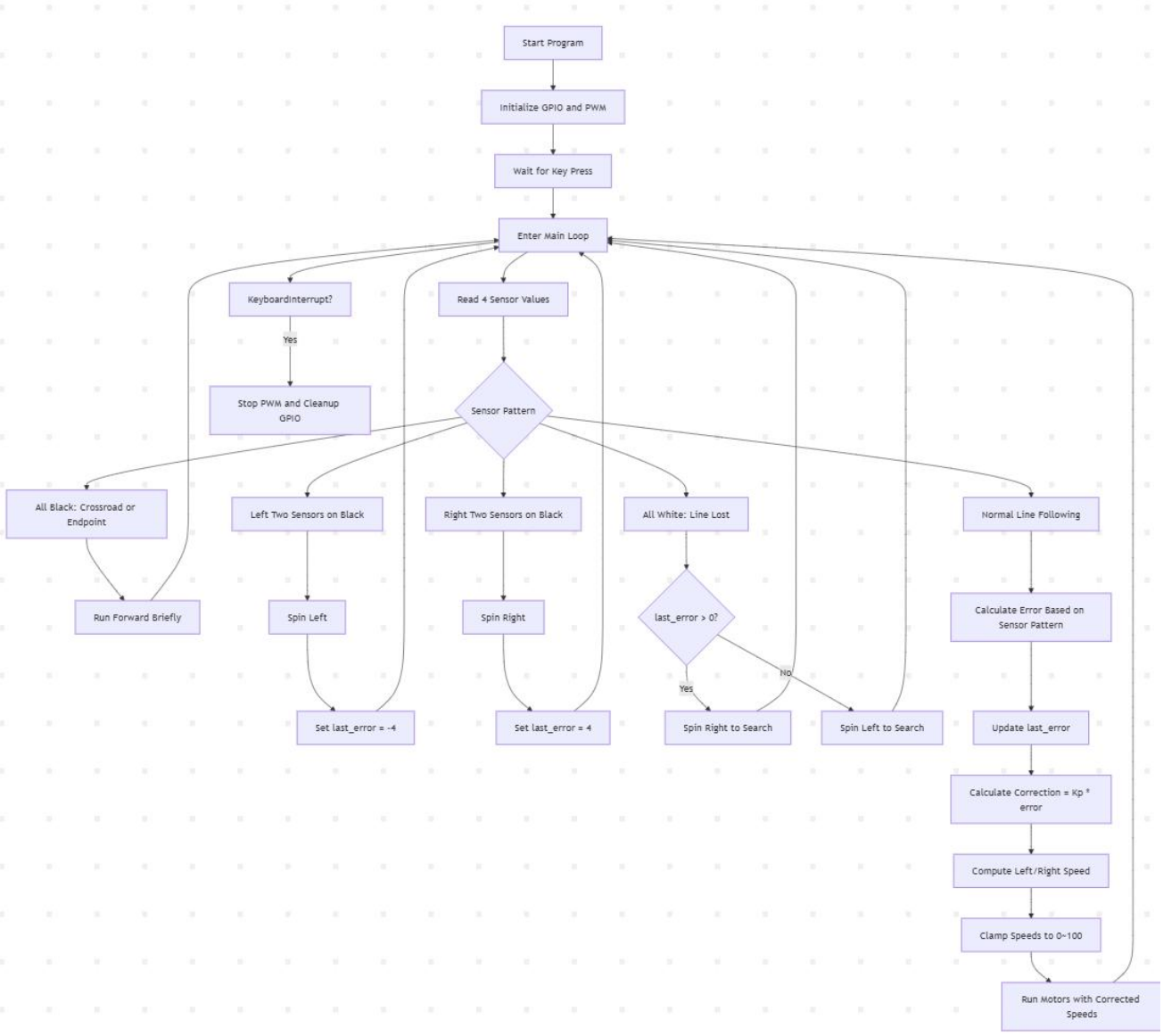
3) Speed limit: Ensure the calculated speed remains within a reasonable range (0-100) to prevent PWM duty cycle overflow.

⑥ **result output**

1) Physical Execution: Unlike the "video writing and display" in machine vision solutions, the "result output" of this solution is physical. The program directly applies the final calculated left speed and right speed to the motors through pwm_ENA.ChangeDutyCycle() and pwm_ENB.ChangeDutyCycle() functions, driving the cart to complete the tracking action.

2) Non-visualization: There is no screen display or file saving of the whole process, and the feedback of the system is the real-time motion trajectory of the cart.

5.3 Flowchart



5.4 Pseudocode

```
WHILE True:
  Read 4 infrared sensor values → sensors[0~3]

  IF sensors == [0, 0, 0, 0]:  # All sensors on black line
    Move forward briefly
    CONTINUE

  ELSE IF sensors[0] == 0 AND sensors[1] == 0:  # Sharp left turn
    Spin left
    last_error = -4
    CONTINUE

  ELSE IF sensors[2] == 0 AND sensors[3] == 0:  # Sharp right turn
    Spin right
    last_error = 4
    CONTINUE

  ELSE IF sensors all white:
    IF last_error > 0:
      Spin right to search
    ELSE:
      Spin left to search
    CONTINUE

  ELSE:
    Calculate error based on sensor pattern:
      error = -1 / 0 / 1 / -2 / 2 depending on deviation
```

Update last_error if error ≠ 0

correction = Kp × error

left_speed  = base_speed + correction

right_speed = base_speed - correction

Clamp speeds to [0, 100]

Run motors with corrected speeds

# 6. Line Follow（camera）

Summarize the line following task.

## a. Sensors

Describe the sensor(s) used to achieve the line following task.

Use a camera to capture the path ahead of the vehicle.

## b. Algorithm

Describe the algorithm used in the line following task.

Firstly, the video data was grayed and Gaussian blurred, and finally converted into black and white images by binarization. Then the image was divided into six lines for centroid calculation. After obtaining the centroid coordinates of the black line, the data was path-fitted.

First, grayscale and Gaussian blur processing are applied to the video data, followed by binarization to convert it into a black-and-white image. Subsequently, the image is segmented into six strips for centroid calculation. After obtaining the centroid coordinates of the black lines, path fitting is performed on the data.

## c. Source Code

Describe the most important functions and blocks of your line following code. DO NOT ADD SOURCE CODE HERE! You can use a **block diagrams** and **flowcharts** to help explaining your code.

Source Code Description for Line Following

This code implements a machine vision-based line detection system that identifies, tracks, and determines the status of a path (black line) by processing video streams, enabling autonomous line-following functionality for Raspberry Pi robots (Homo sapiens). The overall workflow includes video capture, image processing, path analysis, status determination, and result display. The code is primarily divided into two parts:

（1）Image preprocessing function process_for_line_following(): Completes grayscale conversion, blurring, and binarization. The pseudocode is as follows:

- Image Preprocessing Module:
  - Input: Raw BGR frame from camera
  - Output: Binary image with white line on black background

（2）Main loop main(): Responsible for video acquisition, centroid detection, path fitting, direction and curvature determination, offset and lost line detection, as well as video saving and real-time display.

- Main Program:

Initialize camera and PID controller

Set current_mode = 'PID_FOLLOW'

WHILE True:

Read frame from camera

Resize and process image to binary

Analyze line to get state:

- centroids
- turn_size
- turn_direction
- bottom_offset
- lost

```
IF current_mode == 'BIG_TURN_EXECUTE':
    IF turn_context == 'RIGHT':
        IF centroids show left-to-right shift from bottom to top:
            Switch to 'PID_FOLLOW'
        ELSE:
            Execute spin_right with adaptive speed
            CONTINUE loop
    ELSE IF turn_context == 'LEFT':
        IF centroids show right-to-left shift from bottom to top:
            Switch to 'PID_FOLLOW'
        ELSE:
            Execute spin_left with adaptive speed
            CONTINUE loop

IF line is lost:
    CONTINUE loop

IF current_mode == 'PID_FOLLOW':
    IF turn_size == 'BIG':
        Switch to 'BIG_TURN_EXECUTE'
        Record turn_direction and bottom centroid
    ELSE:
        Compute PID correction from bottom_offset
        Adjust motor speeds
```

Run motors

6.1Image Preprocessing Module (process_for_line_following function)

This function serves as the foundation for line-following detection. It enhances the contrast between the "black line" and "background" in the image through a series of image processing operations, providing a clear binary image for subsequent centroid detection. The specific steps are as follows:

(1) Grayscale conversion: Transforming color frames (BGR format) into grayscale images to reduce computational load (cv2.cvtColor).

(2)Gaussian Blur: Apply a 5x5 convolution kernel for Gaussian blur (cv2.GaussianBlur) to smooth image noise and prevent interference from noise points during subsequent binarization.

(3)Binarization: Select between two binarization methods based on the use_otsu parameter:

- Otsu threshold method (default): Automatically calculates the optimal threshold, converting the blurred grayscale image into a binary black-and-white image (black lines as white 255, background as black 0), adapting to varying lighting conditions.
- Adaptive threshold method: Calculates the threshold based on local regions (cv2.adaptiveThreshold), suitable for scenarios with uneven illumination.

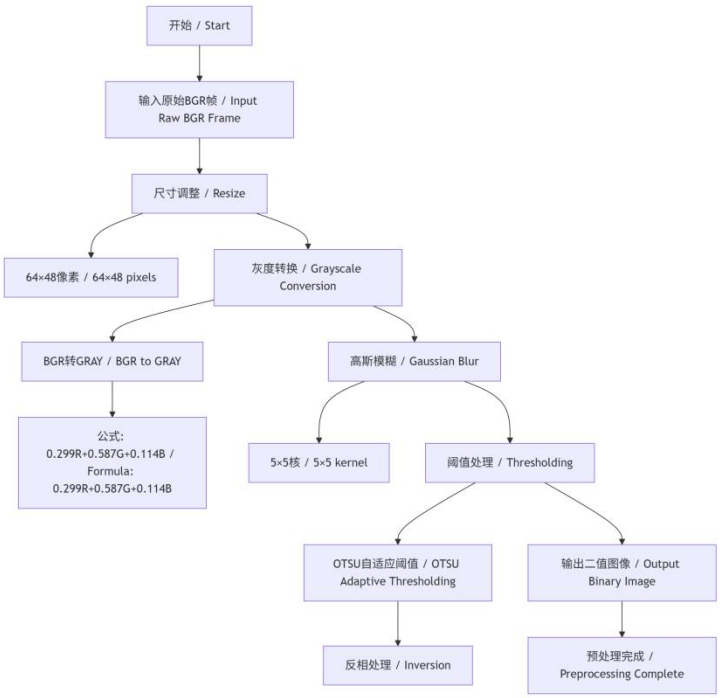



Figure [3] Image preprocessing logic block diagram

## 2.Main logic module (main function)

Responsible for coordinating the entire system workflow, from video stream acquisition to result output.

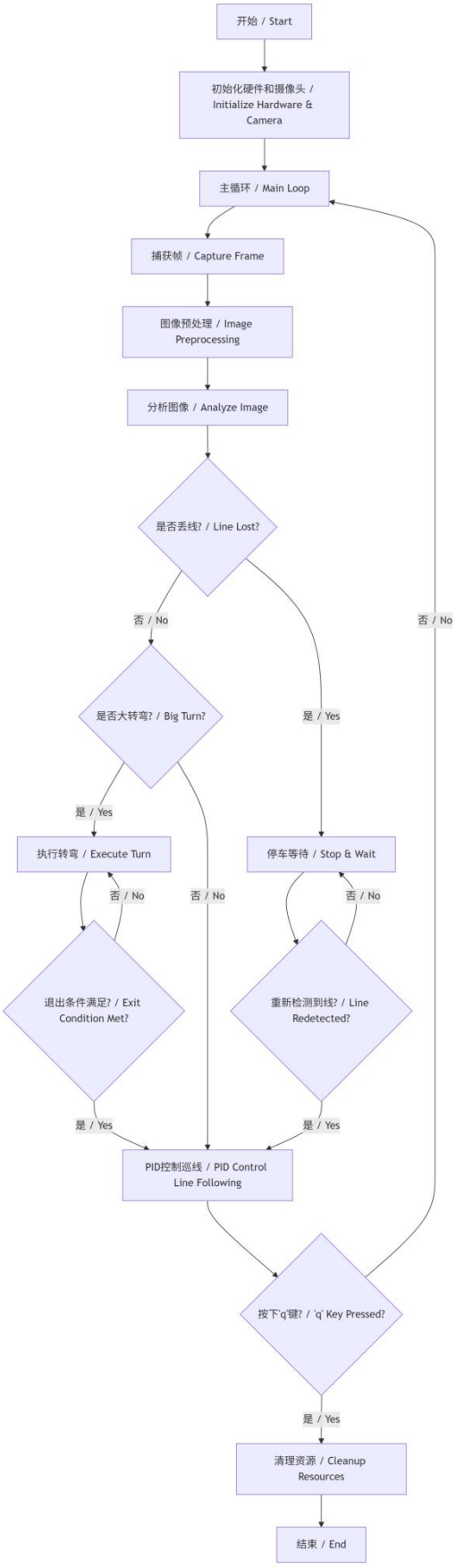



Figure [4] Main Loop Logic Diagram

## 2.1Initial Configuration

(1)Video stream settings: Specify the MJPG-streamer network video stream URL (e.g., http://192.168.50.1:8080/?action=stream).

(2)Output path setting: Define the save paths for the original video and processed video (based on the directory where the script is located).

(3)Video parameter configuration: Force set frame resolution to 128x96, automatically obtain FPS (default 20 FPS), initialize video writer (cv2.VideoWriter).

## 2.2 Video stream acquisition and frame processing

Open video stream: Connect to the network stream via cv2.VideoCapture and wait for the stream to stabilize (2 seconds).

Frame scaling: Scale the read frames to 128x96 to standardize processing dimensions.

## 2.3 Path Centroid Detection

(1)Strip segmentation: First, the frame is vertically divided into 6 strips (num_strips=6), and the black line positions are analyzed strip by strip (simulating a "layered" observation of the path from the perspective of the machine homo sapiens).

(2)Centroid calculation: For each strip of the binarized image, calculate the mean x-coordinate of the white pixels (black line) as the centroid of the black line for that strip (if no white pixels are present, mark it as None).

(3)Path fitting: Connecting the effective centroids of adjacent strips to form a visualized path (linked by yellow line segments).

## 2.4 Direction and Curvature Determination

| 模式/状态 (Mode/State) | 检测条件 (Detection Condition) | 转弯类型 (Turn Type) | 控制策略 (Control Strategy) | 速度参数 (Speed Parameters) | 退出条件 (Exit Condition) |
|---|---|---|---|---|---|
| PID 跟随 (PID Follow) | 检测到大转弯 (Big turn detected) | 大转弯 (Big Turn) | 切换到自适应转弯模式 (Switch to adaptive turn mode) | BASE_SPEED = 10, BIG_TURN_SPEED = 45 | 自动触发模式切换 (Automatically trigger mode switch) |
| PID 跟随 (PID Follow) | 正常巡线状态 (Normal line following) | PID 微调转弯 (PID fine-tuning) | PID 校正控制 (PID correction control) | BASE_SPEED ± PID 校正值 (BASE_SPEED ± PID correction) | 持续 PID 跟随 (Continuous PID following) |
| 大转弯执行 A (Big Turn Execute A) | 右大转弯执行中 (Right big turn executing) | 原地右转 (Spin right) | 根据弯道急缓调整转速 (Adjust speed based on turn sharpness) | 急弯: spin_right(50, 10);<br>缓弯: spin_right(50, 20) | 质心从下往上依次从左往右排列 (Centroids arranged left to right from bottom to top) |
| 大转弯执行 B (Big Turn Execute B) | 左大转弯执行中 (Left big turn executing) | 原地左转 (Spin left) | 根据弯道急缓调整转速 (Adjust speed based on turn sharpness) | 急弯: spin_left(10, 50);<br>缓弯: spin_left(20, 50) | 质心从下往上依次从右往左排列 (Centroids arranged right to left from bottom to top) |
| 丢线状态 (Line Lost) | 绝大多数条带无质心 (Most strips have no centroid) | 停止/刹车 (Stop/Brake) | 停车等待重新寻线 (Stop and wait for re-line finding) | 速度为 0 (Speed = 0) | 重新检测到线路 (Line redetected) |

Table [2] Steering logic judgment table

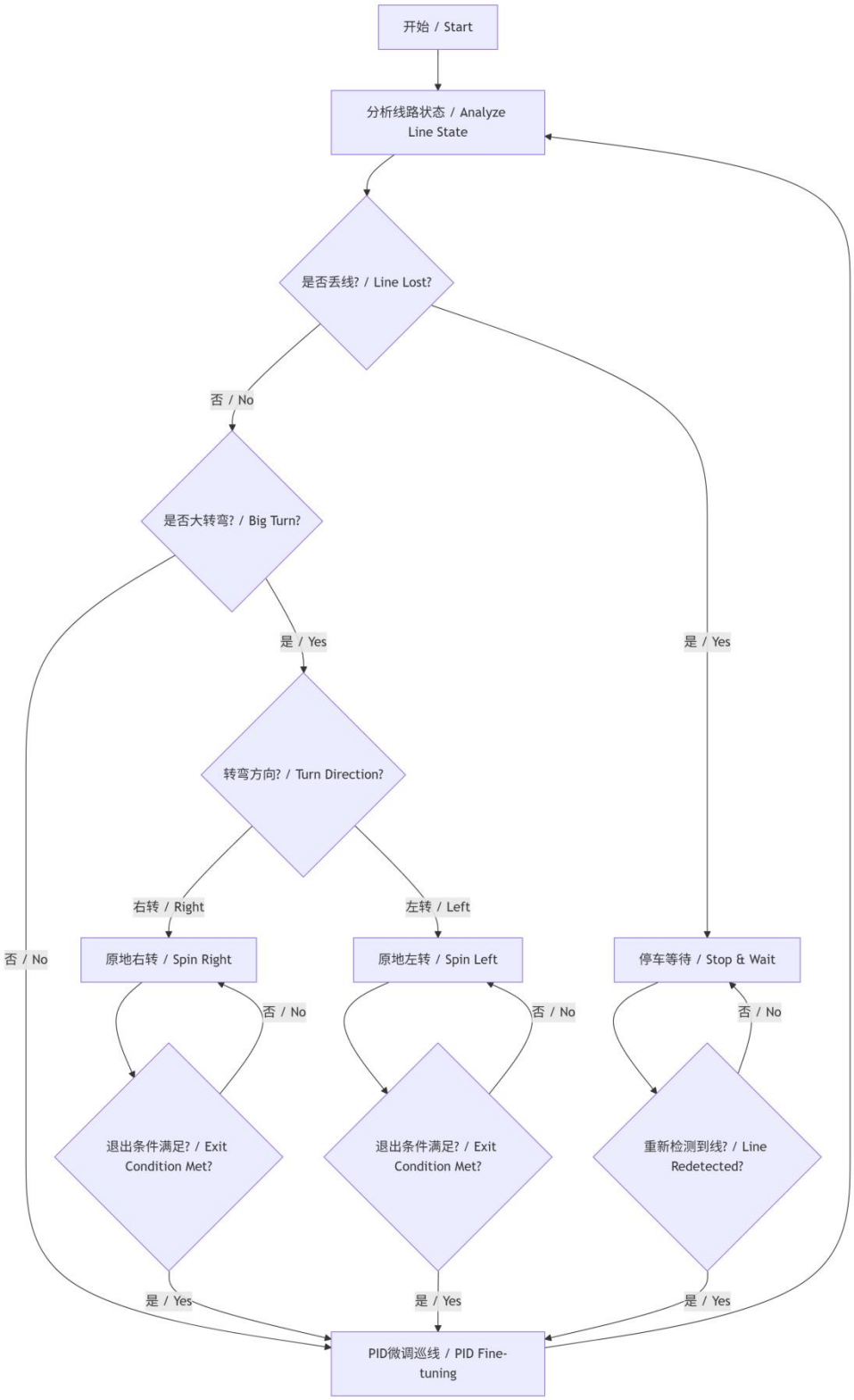



Figure [5] Steering logic block diagram

(1)Direction determination: Based on the lateral variation of waypoints (vector difference among the bottommost, middle, and topmost points), determine whether the path is straight (STRAIGHT), turning left (LEFT), or turning right (RIGHT).

(2)Curvature determination: Calculate the angle (in radians) between path vectors, and classify it as a sharp turn (BIG, >50), gentle turn (SMALL, 10~50), or straight movement (STRAIGHT, <10) based on the angle magnitude.

### 2.5 Offset and line loss detection

(1)Offset calculation: based on the midline of the frame, calculate the lateral offset (move_x) of the nearest effective center of mass and the average offset (move_turn) of all effective center of mass to reflect the degree to which the robot deviates from the path.

(2)Line loss determination: If there are more than 5 strips without valid center of mass (line_none>= num_strips-1), it is determined as line loss (lost=True).

### 2.6 Results display and preservation

(1)Video writing: the original frame (including path tag and status text) is written into output_original.avi, and the binarized frame is written into output_processed_line.avi.

(2)Real-time display: The original frame and processed frame are spliced horizontally, showing FPS, steering information, offset, etc., and exit and release resources by pressing q key.

# 7. Pedestrian Avoidance（sensor）

## a. Sensors

The ultrasonic sensor is used to transmit ultrasonic waves forward and receive the return signal

## b. Algorithm

According to the time of reception, the distance to the front obstacle is calculated. When the distance is less than a certain threshold, the car stops moving forward.

## c. Source Code

**7.1Implementation of a Multi-threaded Line-following and Obstacle-Avoidance Algorithm**

This code demonstrates an advanced robotic control system that concurrently executes two core tasks: trajectory following and obstacle avoidance. By leveraging multi-threading technology, the

system decouples the P-control trajectory-following logic based on four-way infrared sensors from the ultrasonic obstacle detection logic, enabling simultaneous operation. Through shared flag synchronization between threads, the system achieves automatic task suspension upon encountering obstacles during autonomous tracking and resumes operations after obstacle removal. The code is primarily divided into three sections:

- Hardware and Thread Safety Module (init() and motion functions): Handles all hardware initialization and ensures atomicity of underlying motor control through **thread locks (Lock)** to prevent multi-thread conflicts.
- Obstacle Avoidance Thread Module (obstacle_avoidance_thread): An independent execution thread dedicated to continuously detecting obstacles ahead and controlling the robot's start/stop.
- Line Following Thread Module (line_following_thread): Another independent execution thread responsible for implementing the core P proportional tracking algorithm, with its operation being influenced by the status of the obstacle avoidance thread.

### 7.1.1Hardware and thread security module

This module is the foundation of the whole system, not only responsible for hardware initialization, but more importantly, introduces thread security mechanism.

1) init() function:

- Full initialization: In addition to initializing the motor and infrared tracking sensor, the initialization of the ultrasonic sensor (TrigPin, EchoPin) pins is also added.
- PWM Settings: Same as previous versions.

2) Motion function (such as run, brake, etc.):

- Core Improvement-Thread Lock: All operations within the movement function are wrapped in with lock: code blocks. This is a critical design.
- Reason: Since both obstacle avoidance threads and path tracing threads may call functions like brake() or run() to control the motor, the absence of a lock could lead to conflicting instructions between them during execution, causing chaotic GPIO states. Thread locks ensure that only one thread can access and control the motor at any given time, effectively preventing data contention and unpredictable behavior.

## 7.2Core logic module (concurrent execution in two threads)

This is the biggest difference from previous versions. The system assigns two main tasks to two separate threads for parallel processing.

### 7.2.1 Ultrasonic ranging and obstacle avoidance thread (obstacle_avoidance_thread)

The thread's responsibility is very single and clear: safety first. It has the highest priority to control the robot's stop.

1) Data acquisition and filtering:

- The Distance() function triggers ultrasonic waves and calculates distances.
- To enhance data reliability, the Distance_test() function performs 15 consecutive measurements and averages the valid values (excluding erroneous and out-of-range readings). This serves as an effective software filter that significantly reduces misjudgments caused by single-measurement errors.

2) Status determination

- Calculate the average distance between cycles. When 0 <distance <30cm, an obstacle is detected.

3) Core control mechanism-flag bit communication:

- Detecting an obstacle: Immediately set the global shared flag "obstacle Detected" to True. This flag serves as a "pause" signal sent to the tracking thread.
- Brake immediately: call the brake () function to force the car to stop moving.
- Waiting interaction: The program enters a loop, waiting for the user to enter 'q' from the console, simulating the manual confirmation process after removing the obstacle.
- Resume operation: Upon receipt of confirmation, reset obstacle Detected to False and send a "continue" signal to the trace thread.

**7.2.2 Line-following thread (LF)**

This thread is responsible for the core tracing task, but its behavior is limited by the obstacle avoidance thread.

1) Trace algorithm

The internal P ratio control algorithm (including special case processing, error calculation, speed correction, etc.) is exactly the same as the previous "pure trace" version of the code.

2) Key difference-collaboration and concession:

- At the start of each cycle, the system first checks the shared flag: if obstacle detected.
- If obstacle detected is True, the tracking thread abandons all current trajectory calculations and motor control, enters a brief sleep state (time.sleep(0.05)), and initiates the next cycle check. This demonstrates control transfer: when the obstacle avoidance thread sends a pause signal, the tracking thread voluntarily halts its operations, completely transferring control of the motor to the obstacle avoidance thread.

### 7.2.3 State machine and control strategy (multi-threaded version)

The system's state machine now has a clear priority. The obstacle avoidance state is the highest priority state.

| Mode/State (模式/状态) | DetectionCondition(检测条件) | TurnType (转弯类型) | ControlStrategy (控制策略) | SpeedParameters (速度参数) | ExitCondition (退出条件) |
|---|---|---|---|---|---|
| Obstacle Pause (Highest Priority) | obstacle_detected == True (set by obstacle avoidance thread) | Stop / Brake | Call brake(), pause all movement, wait until the flag is reset | Speed = 0 | obstacle_detected == False (reset by obstacle avoidance thread) |
| P-Proportional Line Following | Inner or outer single sensor detects line (and no obstacle) | Proportional fine-tuning turn | P-proportional algorithm correction | base_speed ± (Kp * error) | Switch to another state |
| Sharp Left Turn | Two left sensors detect line (and no obstacle) | In-place left spin | Call spin_left() | spin_left(40, base_speed + 10) | Sensor state changes or obstacle detected |
| Sharp Right Turn | Two right sensors detect line (and no obstacle) | In-place right spin | Call spin_right() | spin_right(base_speed + 10, 40) | Sensor state changes or obstacle detected |
| Intersection / End Point | All four sensors detect line (and no obstacle) | Go straight through | Call run() | run(base_speed, base_speed) | Exit automatically after short delay or obstacle detected |
| Line Lost State | All four sensors detect white (and no obstacle) | Memory-based line search | Turn according to last_error value | spin_left() or spin_right() | Line detected again or obstacle detected |

Table [4] State machine and control strategy (multithreaded version)

### 7.3Main program module

1) Thread management

- Create two thread objects: obstacleThread and lineThread, each bound to their respective functions.
- Set them as daemon threads (daemon = True), meaning these subthreads will terminate automatically when the main program exits, simplifying resource management.
- Launch both threads using the start() method to execute them concurrently.

2) Program life cycle control:

- The main thread maintains its own existence through a while running: loop, providing a runtime environment for child threads.
- It captures user interrupt signals (Ctrl+C) via try...exceptKeyboardInterrupt, setting the running flag to False upon receiving the signal and notifying all child threads that the loop should terminate.

3) Resource cleanup (finally):

- At the end of the program, join() is used to wait for the child thread to exit safely, and then PWM is stopped and GPIO.cleanup() is called to ensure an elegant exit and complete release of resources.

### 7.4 Flowchart

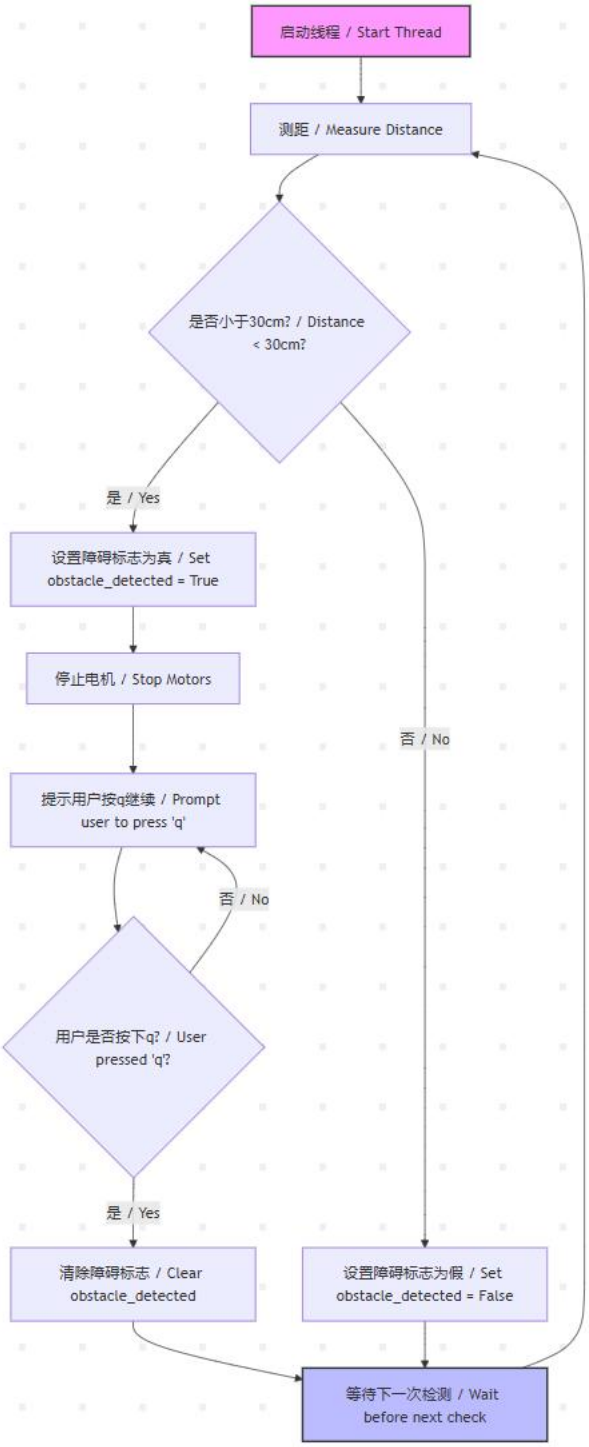



Figure [6] Avoidance logic block diagram

### 7.5 Pseudocode

Function obstacle_avoidance_thread():

Loop while running:

distance = average of multiple ultrasonic measurements

If distance is valid and less than threshold (e.g., 30cm):

Set obstacle_detected to True

Stop motors

Prompt user to press "q" to continue

Wait for user input "q"

Clear obstacle_detected flag

Else:

Set obstacle_detected to False

Wait briefly before next detection

### d. Iteration process

| Version | Behavior | Issue / Reasoning |
|---|---|---|
| VER1 | Robot successfully stops when combining line-following with ultrasonic sensing. | At higher speeds, robot fails to turn in time. Waiting for ultrasonic distance calculations delays line-following response → multithreading needed. |
| VER2 | Robot no longer stops reliably. | Thread priority not handled correctly. |
| VER3 | Robot can stop for obstacles, but inconsistent readings (e.g., dozens show no obstacle, then suddenly one <10 cm). | Likely noisy or faulty data. Averaging multiple readings could help. |
| VER4 (Final) | Stable obstacle detection and response achieved. Unexpected readings from VER3 eliminated. | — |

## 8. Pedestrian Avoidance（camera）

### a. Sensors

Describe the sensor(s) used to achieve the pedestrian avoidance task.

The camera captures the obstacle and sends back the image.

## b. Algorithm

**8.1Describe the algorithm used in the pedestrian avoidance task.**

1)Visual Detection: In its main loop, the program continuously captures the real-time video feed from the camera.

2)Color Recognition: For each image frame, the program calls the has_color() function for analysis. This function is specifically designed to detect the presence of large colored areas in the image.

3)Trigger Condition: If the ratio of colored pixels to the total pixels in the frame exceeds a preset threshold, the program determines that an obstacle has been detected. The code comments specify this is used for "detecting a person."

4)Avoidance Execution: Once an obstacle is identified, the program immediately executes the brake() function to stop the vehicle's motors, and then breaks out of the main loop, ending the entire line-following task.

## c. Source Code

**8.2Implementation of Color Recognition-Based Event Detection and Response Algorithm**

This code snippet demonstrates a machine vision-based event detection module designed to identify preset colors (red, yellow, blue) in camera feeds. Typically deployed as a high-priority interrupt or trigger within main navigation systems, the module activates predefined emergency responses (such as stopping and terminating tasks) when objects matching specified colors (e.g., "robots" or traffic signs) exceed a defined screen proportion threshold. The system architecture consists of two primary components:1

- The core color detection function (has_color) utilizes OpenCV for color space conversion and extraction, serving as the central component of the recognition system.
- The main logic integration module (call section) demonstrates how to invoke the color detection function within the main control loop, enabling high-priority decision-making based on its return results.

**8.2.1 Color Detection Core Module (has_color Function)**

This function encapsulates the complete process of detecting specific colors in a video frame.

1) Image Preprocessing
   - Color Space Conversion: cv2.cvtColor(img, cv2.COLOR_BGR2HSV) Converts the image from BGR to HSV color space.
   - Advantage: HSV separates color (Hue) from brightness (Value), making detection more robust under varying lighting conditions.
2) Target Color Range Definition

- HSV ranges are defined for red, yellow, and blue.
- Special Case – Red: Due to red spanning across the HSV hue boundary (0°/180°), two separate ranges are defined and later merged.

### 8.2.2 Mask Generation and Merging

- Color Extraction: cv2.inRange(hsv, lower_bound, upper_bound) Generates binary masks for each color.
- Mask Merging: mask = mask_red | mask_yellow | mask_blue Combines all masks into one, highlighting all target-colored pixels.

### 8.2.3 Decision Logic and Thresholding

- Area Calculation: np.count_nonzero(mask) counts white pixels in the merged mask.
- Color Ratio: Divides the color pixel count by total frame pixels to get color_ratio.
- Return Condition: return color_ratio > threshold (default threshold = 2%) Ensures

## 8.3 Flowchart

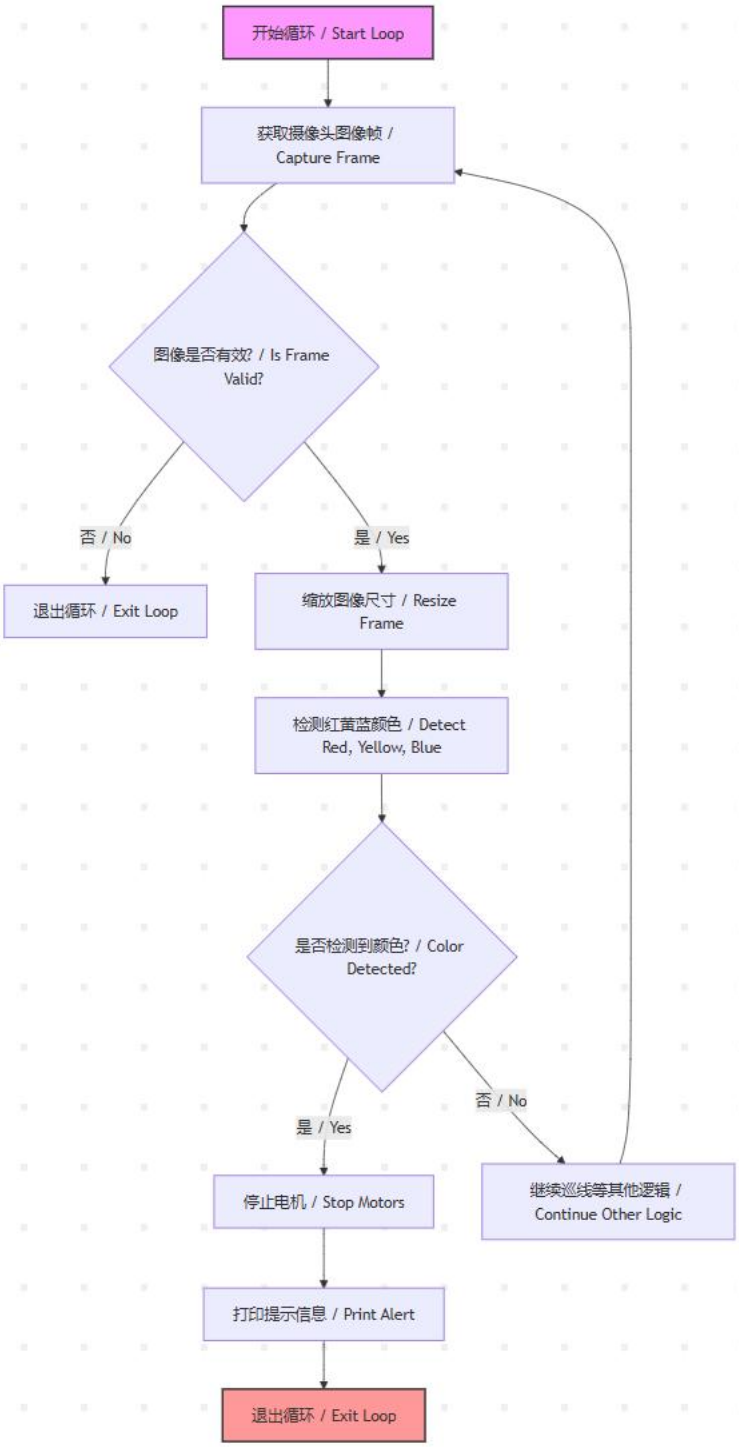



Figure [7] Vision Avoidance logic block diagram

**8.4 Pseudocode**

Function camera_obstacle_avoidance():

Loop while running:

Capture frame from camera

If frame is invalid:

    Exit loop

Resize frame

Check for red, yellow, or blue color

If color is detected:

    Stop motors

    Print alert

    Exit loop

Else:

    Continue with other logic (e.g., line following)

## d. iteration process

| Version | Implementation / Improvement | Behavior | Reasoning |
|---|---|---|---|
| v1 | Defined curve detection function, integrated with PID. | Follows straight/curved paths, fails on sharp turns. | Curve analysis logic too simplistic. |
| v2 | Refined logic to distinguish small vs. large turns. | Large turns detected, but turning performance poor. | Parameters not well tuned. |
| v3 | Adjusted rotation parameters. | Turning unstable; robot may spin in place. | Camera misinterprets right turns as left; flawed rotation logic. |
| v4 | Modified exit condition for large-turn loops. Logic: analyze centroids of bottom stripes; adjust turn angle; maintain right-turn state if leftward shift; exit only on small-turn/straight detection. | Still fails to turn correctly. | Exit condition too idealistic, needs refinement. |
| v5 | Changed exit condition: requires consistent centroid shifts across multiple stripes. | Turning more reliable, but sharp/right-angle turns unstable. | Need to distinguish sharp vs. right-angle turns within “large turn.” |
| v6 (Final) | Added logic to identify right-angle turns within large-turn scenarios. | Stable recognition of right-angle turns. | — |

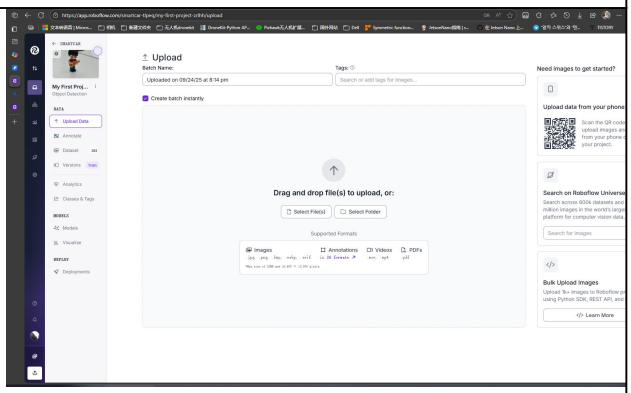
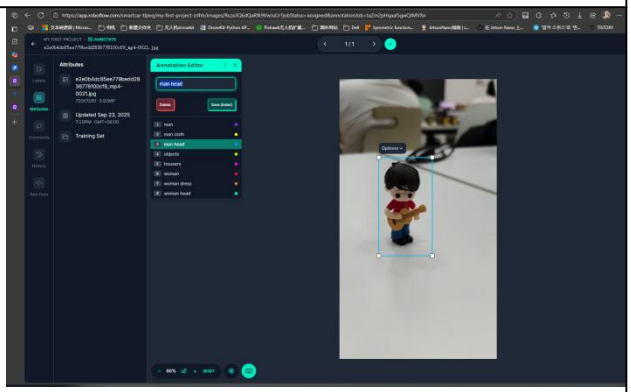
# 9.Deep Learning – Detection & Classification（YOLO）

| Subtask | Tool/Method | Description |
|---|---|---|
| Dataset Preparation | Roboflow: Computer vision tools for developers and enterprises | Image collection, annotation, augmentation, YOLO export |
| Model Training | YOLOv11 | Train detection model with labeled dataset |
| Detection Pipeline (Inference) | Raspberry Pi + MJPG-Streamer | Provide real-time video feed via HTTP stream |

Table [5] Task splitting

## 9.1Dataset Preparation (Roboflow)

In this project, we used Roboflow: Computer vision tools for developers and enterprises **online platform** to build and manage the dataset. The detailed steps are as follows:

| Step | Content | Description | Image |
|---|---|---|---|
| 1. Data Source | Collect raw data via onboard camera, including **video streams and static images** | Provide diverse training samples covering different scenarios |  |
| 2. Upload | Upload collected videos and images to the Roboflow platform | Centralized management for annotation and processing |   |
| 3. Annotation | Manually annotate bounding boxes for target objects (e.g., pedestrians, obstacles, traffic signs) | Define detection categories and generate label files |  |

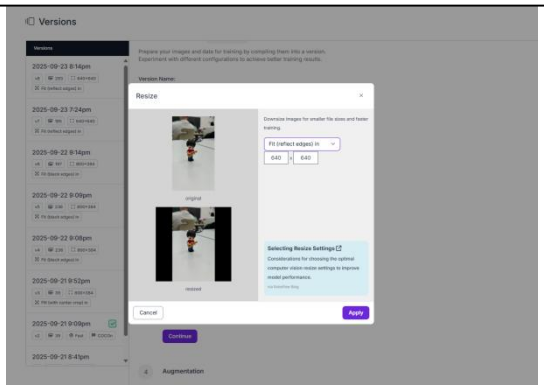
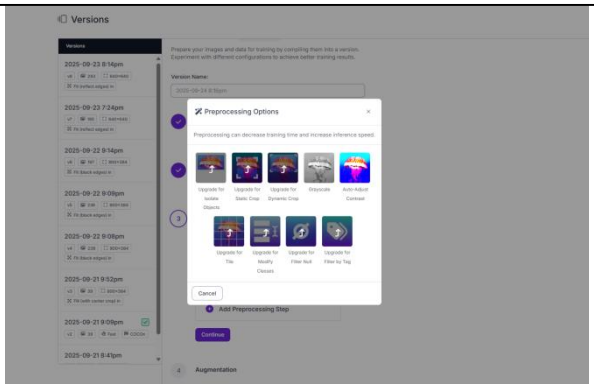
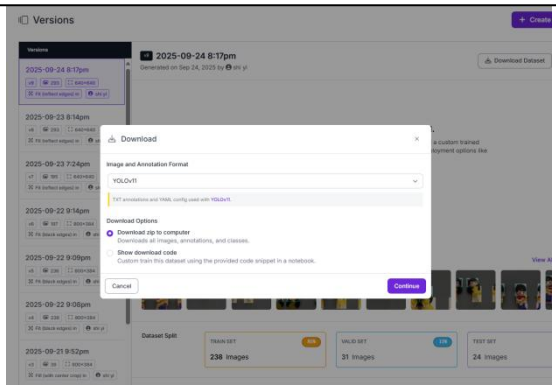
| Step | Content | Description | Image |
|---|---|---|---|
| 4. Preprocessing | Use Roboflow tools for resizing (e.g., 640×640) and format conversion | Ensure YOLO compatibility and unified input size |  |
| 5. Augmentation | Apply rotation, brightness adjustment, flipping, etc. | Increase dataset diversity and model robustness |  |
| 6. Export | Export dataset in **YOLO format** for YOLOv11 training | Directly compatible with training framework |  |

Table [6] Step diagram

## 9.2Model Training (YOLOv11)

| Step | Content | Description |
|---|---|---|
| 1. Environment Setup | **Hardware**: GPU server or local CUDA-enabled GPU (e.g., NVIDIA RTX)<br>**Software**: Python 3.8+, PyTorch (GPU), OpenCV, NumPy, YOLOv11 repo<br>**Dataset**: YOLO-format dataset exported from Roboflow (images/, labels/, data.yaml) | Ensure environment and dataset readiness |
| 2. Parameter Config | **Input size**: 640×640<br>**Batch size**: 0.8<br>**Epochs**: 400<br>**Optimizer**: SGD or AdamW<br>**Learning rate**: default 0.01, adjustable<br>**Loss functions**: IoU-based box regression, objectness loss, classification loss | Control training process and ensure convergence |
| 3. Training Execution | 1) Load dataset via data.yaml<br>2) Load model with pretrained weights (e.g., yolov11s.pt)<br>3) Start training: monitor loss curves and mAP<br>4) Validate model on validation set | Execute training and monitor performance |
| 4. Model Saving & Export | **Save weights**: best model saved as best.pt<br>**Export**: ONNX or TorchScript for deployment on Raspberry Pi or embedded devices | Ensure reusability and deployment readiness |

Table [7] Step diagram of Model Training

## 9.3Training Results Comparison

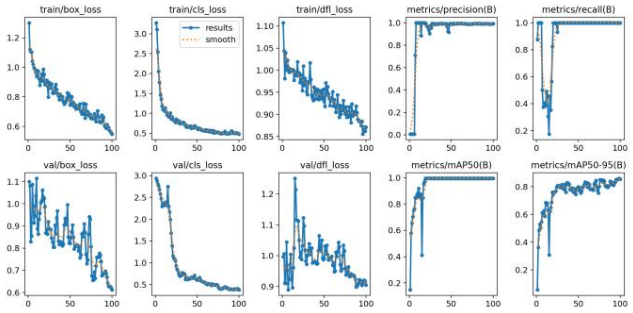
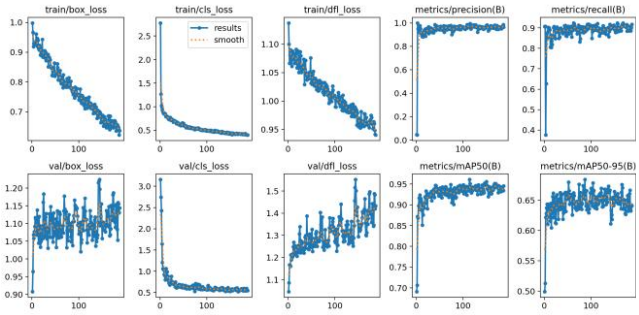
| Model Setup | Classes | Training Result Image | Metrics | Analysis |
|---|---|---|---|---|
| **Option 1: Detect only man / woman** | 2 |  | - Precision: High<br>- Recall: High<br>- mAP@0.5: ≈0.95<br>- mAP@0.5:0.95: ≈0.82 | Fewer classes, easier convergence, faster inference, higher accuracy. Cannot distinguish fine-grained features (e.g., clothing, head). |
| **Option 2: Detect man_head / man_cloth / trousers / woman_head / woman_dress** | 5 |  | - Precision: Medium<br>- Recall: Medium<br>- mAP@0.5: ≈0.90<br>- mAP@0.5:0.95: ≈0.75 | More detailed classes, harder training, slightly lower accuracy. Provides richer detection information for fine-grained tasks. |

Table [8] Training Results

## 9.4Detection & Classification Pipeline – Inference

| Step | Content | Description |
|---|---|---|
| 1. Video Stream | Use MJPG-Streamer to provide HTTP video stream (e.g., http://192.168.50.1:8080/?action=stream), capture via OpenCV | Fallback to local camera if remote stream unavailable |
| 2. Model Loading | Load YOLOv11 weights (e.g., best.pt), prefer GPU (CUDA), fallback to CPU | Use ultralytics.YOLO API to load and set device |
| 3. Model Selection | Choose model based on task:<br>① **Detection Model** (man / woman)<br>② **Classification Model** (man_head / man_cloth / trousers / woman_head / woman_dress) | Controlled via config or parameters for flexible granularity |
| 4. Inference | Run YOLOv11 on each frame with confidence threshold (e.g., conf=0.25) | Output bounding boxes, class IDs, and confidence scores |
| 5. Visualization | Use OpenCV to draw bounding boxes and labels | Green rectangles, label backgrounds, text annotations |
| 6. Performance | Calculate per-frame inference time and estimate FPS with exponential smoothing | Displayed on top-left corner for performance evaluation |
| 7. Video Saving | Save processed frames to local video file (e.g., yolo_output3.avi) | Use cv2.VideoWriter to save detection results |
| 8. Exit | Press q or trigger KeyboardInterrupt to exit loop, release resources | Close camera, writer, windows, and print save path |

## 9.5Flowchart

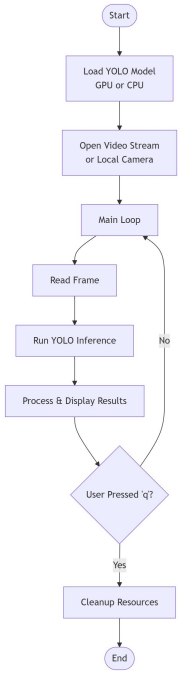



Figure [8] YOLO Flowchart

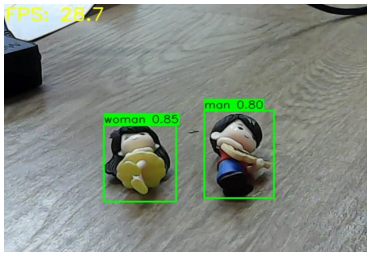
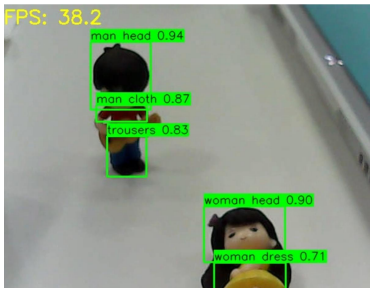
## 9.6 Pseudocode

```
function main():
  try:
    if GPU is available:
      load YOLO model on GPU
    else:
      load YOLO model on CPU
    try:
      open video stream
    except:
      try local camera
      if still fail:
        exit program
    while True:
      try:
        read frame
        if frame is valid:
          run inference
          draw results
          show and save frame
        else:
          continue
      except:
```

```
            handle inference error
        if user pressed 'q':
            break
    except:
        handle unexpected error
    finally:
        release video and writer
        close windows
```

### Inference Results

## Inference Results Comparison

| Model Type | Class Setup | Example Detection Result | Metrics | Analysis |
|---|---|---|---|---|
| **Detection Model** | 2 classes: man / woman |  | - Precision: ≈0.93<br>- Recall: ≈0.90<br>- mAP@0.5: ≈0.95<br>- mAP@0.5:0.95: ≈0.82 | Only distinguishes man / woman. Fast convergence, high accuracy, suitable for quick detection tasks. Cannot provide fine-grained details. |
| **Classification Model** | 5 classes: man_head / man_cloth / trousers / woman_head / woman_dress |  | - Precision: ≈0.88<br>- Recall: ≈0.85<br>- mAP@0.5: ≈0.80<br>- mAP@0.5:0.95: ≈0.75 | More detailed categories, harder training, slightly lower accuracy. Provides richer information, suitable for fine-grained recognition tasks. |

# 9. Source Files and Videos

| 模块 (Module) | 文件 / 子目录 (Files / Subfolders) |
|---|---|
| **structure** | structure.txt |
| **1. sensor_line_follow** | ★sensor_line_follow.py |
| **2. Camera-Line-Following** | ★Camera-Line-Following.py |
| **3. Ultrasonic Obstacle Avoidance** | ★Ultrasonic Obstacle Avoidance.py |
| **4. Vision Obstacle Avoidance** | ★Vision Obstacle Avoidance.py |
| **5. Deep Learning – Detection & Classification** | 1. Classification (man_head / man_cloth / trousers / woman_head / woman_dress)<br>● datasetPreparation → Dataset((man_headman_clothtrouserswoman_headwoman_dress))<br>● Predict → predict.py<br>● trainModel → bestModel.pt, train.py<br>2. Detection (man / woman)<br>● datasetPreparation → Dataset(man_woman)<br>● Predict → predict.py<br>● trainModel → bestModelt.pt |
| **6. demo_video** | 1.★sensor_line_follow.mp4<br>2.★Camera-Line-Following.mp4<br>3.★SensorUltrasonic Obstacle Avoidance.mp4<br>4.★Vision Obstacle Avoidance.mp4<br>5.★Deep Learning-Detection(man_woman).mp4<br>6.★Deep Learning – Classification((man_headman_clothtrouserswoman_headwoman_dress)).mp4 |